

Advanced Operating Systems

Indice

Intro	3
Processes	3
TCB	3
Memory maps	4
Task state	4
Wait queues	4
Task hierarchies	5
Task scheduling theory	5
Task model	6
Platform model	6
Problem statement	6
Metrics	6
Algorithm classification	7
Scheduling algorithms	7
FIFO	7
Shortest Job First (SJF)	7
Shortest Remaining Time First (SRTF)	7
Highest Response Ratio Next (HRRN)	7
Round Robin	8
Priority-based scheduling	8
Multi-level Queue Scheduling	8
Multi-level Feedback Queue Scheduling	8
Multi-level Feedback Queue Scheduling with time slicing	9
Multi-level Feedback Queue Scheduling with aging	9
Multi-processor scheduling	9
Single vs multiple queues	9
Scheduling in Linux	9
Runqueues	9
Scheduling classes	10
CFS	10
Cgroups	10
Load balancing	10
IPC	11
Fork-wait	11
Signals	12
Unnamed pipes	13
Named pipes (FIFO)	13
Message queues	13
Shared memory	14
Synchronization: semaphores	15

Concurrency	16
Issues	16
Kernel space concurrency	16
Kernel preemption	17
Synchronization	17
rwlock	17
seqlock	18
Sleeping lock	18
lockdep	18
Cache aware spinlocks	19
Lock-free algorithms and data structures	19
Per-CPU variables	19
Shared variables	20
Read-copy-update	20
Memory consistency models	21
Sequential consistency	21
Total store order	22
Partial store order	22
Hardware synchronization techniques	22
The linux kernel memory model (LKMM)	22
Linux memory management	23
Process address space	23
Kernel address space	24
Kernel space allocation	24
The slab allocator	24
The buddy allocator	25
Physical address space	25
Zonal page allocation	25
User-space page caching	25
Page frame reclaim	26
System virtualisation	26
Software based virtualisation	27
Problems with pure trap-and-emulate on Intel machine	27
Hardware assisted virtualisation	28
Intel VT-x	28
KVM	28
Containerization	29
IO and peripherals	29
CPU to device interactions	29
Device to CPU communication	30
Linux low-level IO	30
Interrupt management	30
Tasklets	31
Work queues	32
Devices in Linux	32
Block devices	32
The linux device model	32
The platform bus framework	33
The PCI bus framework	33
Linux platform configuration and boot	33
ACPI	33

Intro

An OS provides:

- **Resource management:** by allowing programs to run as if they were assigned their individual resources and by multiplexing/virtualising access to ensure fair utilization of resources
- **Isolation and protection:**
 - Regulating/enforcing access rights to resources to avoid conflicts
 - Enforcing data access rights
 - Ensure mutual exclusion when needed
- **Portability:** by hiding the complexity of hardware access and allowing the same applications to work on systems equipped with different physical resources
- **Extensibility:** interface/implementation abstractions to allow adding new components and hiding complexity associated to variants

Processes

TCB

The **task** is the common denominator between a thread and a unix process (which are basically processes without threads). It contains:

1. A unique **program counter**
2. **Two stacks**, one in user mode and one in kernel mode (2 pages wide)
3. A set of **registers**
4. An **address space** (for kernel mode this is the whole kernel address space)

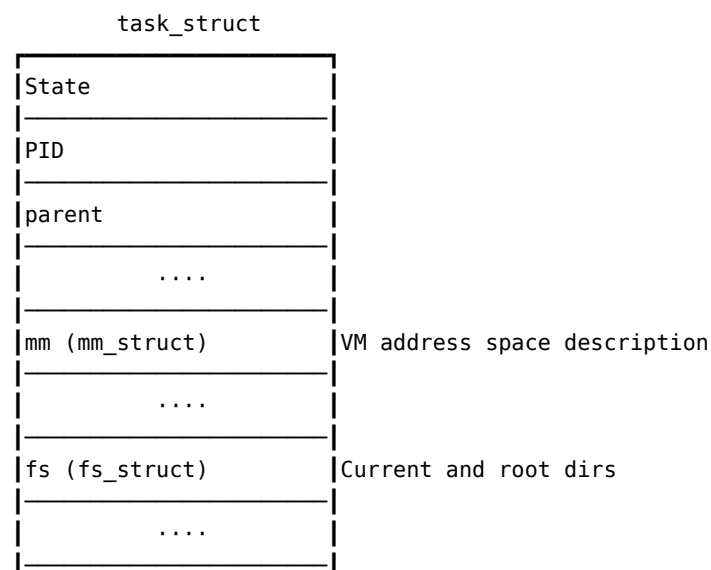
`task_struct`, or the Task Control Block (TCB), is the C struct that contains all of the data about a given task. The **TCB is doubly linked to the `thread_info` structure (one per CPU), which resides at the bottom of the process's kernel stack**. This structure is used to **move between TCB and kernel stack**:

1. The TCB contains a** pointer to its** `thread_info` (called `*stack`)
 - Due to how `thread_info` is located, `*stack` is the **bottom of the kernel stack**
2. `thread_info` contains a **pointer** to its `task_struct`

One very important variable contained in `thread_info` is `preempt_count`, which will be seen when we talk about kernel concurrency. `thread_info` is **architecture specific**.

NOTE: On newer kernels, the link between `task_struct` and `thread_info` is different.

When the kernel does a context switch, the **preserved registers will be saved in** a structure called `thread_struct`.



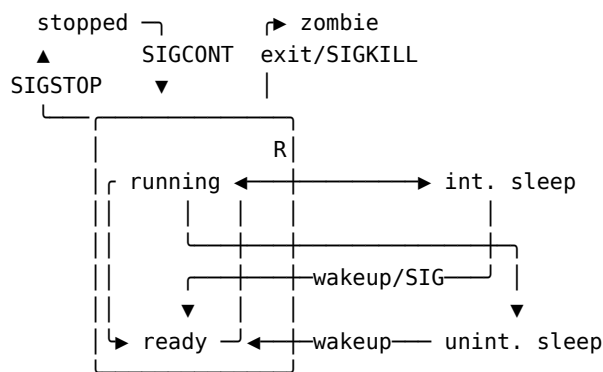
files (files_struct)	Opened files
....	
signal (signal_struct)	Signal info
....	
thread_struct	\$sp, \$ra and callee saved regs at context switch

Memory maps

The most important memory maps are:

- start_stack
- mmap_base: start of memory-mapped areas
- start_brk and brk: respectively start and end of heap
- start_data and end_data
- start_code

Task state



Wait queues

// Source @ <https://elixir.bootlin.com/linux/v6.1.5/source/include/linux/wait.h#L30>

```

struct wait_queue_entry {
    unsigned int    flags;
    void           *private; // *task_struct
    wait_queue_func_t func;
    struct list_head entry;
};

struct wait_queue_head {
    spinlock_t      lock;
    struct list_head head;
};

typedef struct wait_queue_head wait_queue_head_t;

struct task_struct;
  
```

Each task that is sleeping is waiting for an event. **Each event has a wait queue and tasks enqueue themselves into those queues** by calling either `wait_event()` and `wait_event_interruptible()` and passing a callback (`wait_queue_func_t`).

Events can wake up all the threads in the queue by calling `wake_up()` and invoke their callback. This creates a “thundering herd”: only some tasks will be able to read the data, all the others will simply wake up and then go back to sleep.

The solution is to **divide the queue of tasks into two parts**:

1. **Exclusive**: new tasks are put at the end of the queue
2. **Non-exclusive**: normal tasks

The kernel will **wake up all non-exclusive tasks but only the first exclusive one**.

Task hierarchies

We can **create new tasks** by calling `fork()`, which **internally invokes `sys_clone` which creates a new copy of the `task_struct` struct**. The fork-ed process will be called a ‘child’ while the fork-ing process the parent.

The copy will differ from the parent in terms of:

1. **PID and PPID** (parent PID)
2. **Certain resources**, such as pending signals, **which are not inherited**

Rather than duplicating the process address space, the parent and the child can **share the same copy**. If data is written to the shared copy, a duplicate is made and each process receives a unique copy (**CoW** i.e. Copy on Write)

The **init task is the first process** (pid 0). After **initializing the kernel** it creates a **new kernel thread** (pid 1) which will:

- **Finish starting the kernel**
- **Load the `initrd` in memory**
- **`kernel_execve` to `/bin/init`** (the system’s init system)

To get things into motion, **pid 0 will call the first `schedule()`** so that pid 1 can start executing.

The **traditional init system was SystemV**, or `sysv`. In `sysv`, the **configuration of the system is divided in “runlevels”** which are different configurations of the system (single-user, single-user with network etc...). **The definition of what each runlevel runs is specified in `/etc/inittab`**. Each runlevel was **basically a script that run different modules sequentially**.

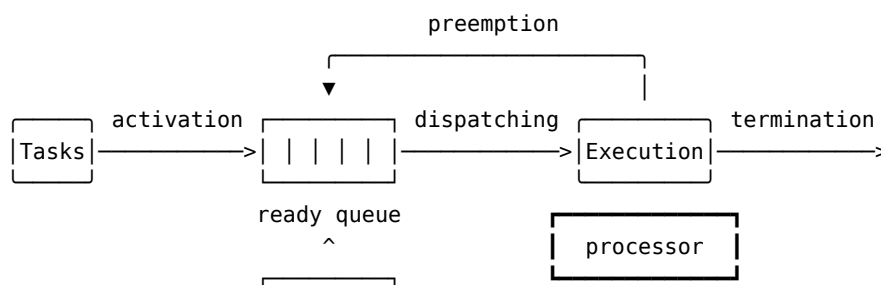
The **modern (and normal) init system is SystemD**. SystemD **overcomes the limitation of sequential execution and also uses a declarative model** (unit files) instead of the imperative one (scripts) of `sysv`:

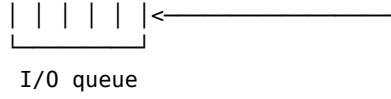
1. The init daemon **loads various units**, which are **plain-text files that contain information about the service**
 - Services are the most common type of units, but not the only one
 - They **contain the ordering** to ensure that dependencies are started before the requiring unit
2. **Targets are collections of units** and emulate old runlevels

Task scheduling theory

A task, in this context, can be thought of as a **synonymous of a thread**. However remember that it has no fixed definition: it is **simply the basic scheduling unit**.

A **scheduler** is the OS component in charge of **establishing the execution order of tasks**. The **ordering algorithm** is the **scheduling policy**.





Preemption is the operation of **temporarily suspending the execution of a task in order to execute another task**. Can be task-triggered (unusual) or OS-triggered (more common). Preemption is performed via a context switch.

Task model

We can model a task i with the following **parameters**:

- a_i : **Arrival time** (or Request time); the time instant at which task is ready for execution and put into the ready queue
- s_i : **Start time**; the time instant at which execution actually starts
- W_i : **Waiting time**; the time spent waiting in the ready queue ($W_i = s_i - a_i$)
- f_i : **Finishing time**; time instant at which execution terminates
- C_i : **Computation time**; amount of time necessary for the processor to execute the task without interruptions
- Z_i : **Turnaround time**; difference between finishing and arrival time (not necessarily $W_i + C_i$)

In case of preemption/suspension Z_i also contains interferences from other tasks and the time the task is interrupted

Depending on the type of operations dominating the lifetime of a task, we may identify **two bounds** for a task:

- **CPU-bound**: spends most of the time executing
 - $Z_i \approx W_i + C_i$ excluding preemptions
- **I/O-bound**: spends most of the time waiting for I/O
 - $Z_i \gg W_i + C_i$ excluding preemptions

Platform model

A computing system is composed of:

1. m **processing elements** (PEs) $\{CPU_1, \dots, CPU_m\}$
 - Each PE, at each t , is assigned zero or one task $A_{cpu}(CPU_k, t) = \tau_i \vee \emptyset$
 - A task τ_i can execute at time t only if $\exists A(CPU_k, t) = \tau_i$
2. s **additional resources** $\{R_1, \dots, R_s\}$
 - Each resource, at each t , is assigned to zero or more tasks: $A_{cpu}(CPU_k, t) = \{\tau_i, \tau_j, \dots\} \vee \emptyset$
 - Depending on the type of resource, it may be exclusive
 - A task τ_i can execute at time t if $\tau_i \in A(R_k, t)$ for all R_k requires to run the task

Problem statement

Given:

1. A set of n tasks T
2. A set of m PEs
3. A set of resources
4. (Optionally) A set of precedent relationships and constraints

Compute an optimal schedule and allocations.

It is a **NP-complete problem** (usually reduced to the knapsack problem).

Metrics

The scheduler aims at **optimizing one or more objectives**. Some metrics that help to determine if a policy is good are:

- **Processor utilization**: percentage of time the CPU is busy
- **Throughput**: number of tasks completing their execution per time unit
- **Waiting time**: time the tasks spends in the ready queue

- **Fairness:** do the tasks have a fair allocation of processor resources?
- **Overhead:** amount of time spent taking scheduling decisions and context-switches
- **Energy, power, temperature** and many more

Of course, some **trade-offs** have to be made since:

- We want to maximize utilization, throughput and fairness
- While we minimize the turnaround, waiting and completion time and overhead

And these goals clash with each other.

Whatever is the algorithm/policy, **the scheduler must guarantee that all tasks are served. Starvation is the perpetuated condition where one or more tasks cannot execute due to the lack of resources.**

Algorithm classification

- **Preemptive vs non preemptive**
 - Preemptive: tasks can be interrupted by the OS at any time to make room for other tasks
 - Non-preemptive: once started, tasks are executed to completion, guaranteeing the lowest overhead
- **Static vs dynamic**
 - Static: scheduling decisions are based on fixed parameters
 - Dynamic: scheduling decisions are based on parameters that change at runtime
- **Offline vs online**
 - Offline: the scheduler is executed on a set of known tasks before their activation, the output is the sequence of tasks (the scheduler must also be static)
 - Online: the scheduler executes at runtime
- **Optimal vs heuristic**
 - Optimal: based on an algorithm optimizing a given cost (high overhead)
 - Heuristic: based on heuristic functions

Scheduling algorithms

FIFO

Also known as First Come First Serve (FCFS), tasks are **scheduled in the order of arrival**. It is **non-preemptive**, very simple and does not require any knowledge of the process. However it is **terrible for responsiveness**.

Shortest Job First (SJF)

Also known as Shortest Job Next (SJN), tasks are **scheduled in ascending order of computation time (C_i)**. Like FIFO it is also a **non-preemptive** algorithm.

It is the **optimal** (proof in the slides) **non-preemptive algorithm w.r.t minimizing the average waiting time**. The **problems** are mainly two:

1. We run into the risk of **starvation** for long tasks
2. We **need to know C_i in advance**

Shortest Remaining Time First (SRTF)

It is the **preemptive variant of SJF**. It **uses the remaining execution time instead of C_i** to decide which task to dispatch.

The advantages it provides is **an improvement in responsiveness** compared to SJF, but it **does not solve any of its drawbacks** (i.e. risk of starvation for long tasks and the need of C_i).

Highest Response Ratio Next (HRRN)

Selects the task with the highest Response Ration:

$$RR_i = \frac{W_i + C_i}{C_i}$$

It is **non-preemptive** and **prevents the starvation** that SJF may cause. We still **need to know C_i in advance**.

Round Robin

Tasks are scheduled for a given time quantum q (also called time slice). When the **time quantum expires**, the task is **preempted and moved back to the ready queue**.

Advantages:

1. **Computable maximum waiting time:** $(n - 1) * q$
2. **No need to know C_i in advance**
3. **Good to achieve the fairness and responsiveness goals**
4. **No starvation** is possible

Disadvantage: **turnaround is worse** than SJF.

In RR the choice of the time quantum is important:

- **Long quantum favors CPU-bound tasks** and reduces overhead, the scheduling tends to FIFO as the quantum increases
- **Short quantum favors IO-bound tasks** and reduces average waiting time, making it better for responsiveness and fair scheduling; it has, however, higher overhead due to more context switches

Priority-based scheduling

The **priority P_i** is a **task parameter** through which we can **specify the importance of a task**. The priority can be **fixed or dynamic** and is usually expressed with an **integer value** with the following convention:

- The **lower** the integer value, the **higher the priority**
- The **higher** the integer value, the **lower the priority**

Multi-level Queue Scheduling

We have **different run-queues, each containing tasks of different priority**. This also provides **quick task ordering** based on priority. **Each queue can specify a different scheduling algorithm**. The **first task to schedule is picked from the topmost non-empty queue** (highest priority). Tasks **cannot be moved from one run-queue to another**.

This method incurs the **risk of starvation**: while the higher priority run-queues fill up, we delay the lower priority tasks.

A simple scheme is using **RR on each queue and varying the time quantum**: **smaller quantum for high priority** and **larger quantum for lower priority**. Priority measure then becomes a measure of the boundness of the process:

- **CPU-bound tasks are lower priority** since they benefit from smaller quanta
- **IO-bound tasks are higher priority** since they benefit from larger quanta

This scheme **guarantees the best responsiveness** (without dealing with starvation). To determine whether a task is **CPU/IO-bound** we need that the **information is delivered**:

- By the **user**
- By the program itself at **runtime** (but it requires a feedback mechanism)

Multi-level Feedback Queue Scheduling

The scheduling works like the previous multi-level RR scheme. The **priority is now dynamic**, changing according to this rationale:

1. The **new/activated task is moved to the highest priority queue**
2. If the **quantum of the running task expires**, the task is **moved to the next queue with lower priority**

Thus **CPU-bound tasks are progressively moved in queues with longer time quantum**. We, however, still **haven't solved the problem of starvation**.

Multi-level Feedback Queue Scheduling with time slicing

Each queue gets a maximum percentage of the available CPU time it can use to schedule the task, which determines a time quota. If the time quota expires, the remaining tasks in the queue are skipped, and we start picking tasks from the next (lower priority) queue.

The sum of all the quotas can be greater than the total period, but the absence of starvation is guaranteed only if the sum of quotas does not exceed the period. We can still have starvation due to the scheduling policies of one of the queues.

A similar schema is used by the Linux kernel.

Multi-level Feedback Queue Scheduling with aging

The priority of the task is increased as long as it spends time in the ready queue (it gets older). This prevents a task from being indefinitely postponed by new incoming higher priority tasks, thus avoiding starvation.

Multi-processor scheduling

In multi-processor systems we need to choose the task to execute and to which processor to assign it. This model brings more problems:

1. **Task synchronization may occur** across parallel executions
2. It is difficult to **fully utilize all processors**
3. **Simultaneous** (not only concurrent) **access to shared resources**

Lower level **cache conflicts between processes are one of the big problems** that scheduling needs to consider.

We can have **two main design choices**:

1. **Single queue vs multiple queues**
2. **Single scheduler vs multiple per-processor schedulers**

Single vs multiple queues

In the **single queue** all the ready task wait in the same global queue. This is **simpler**, good for fairness and managing CPU utilization, but it **doesn't scale well** since the queue **needs to be synchronized**.

In the **multiple queues** case we **assign a ready queue for each processor**. This is **more scalable** and **better leverages data locality**, with potentially **more overhead**. It also **allows the use of both a global scheduler or per-CPU schedulers**. It needs to implement a **load-balancing scheme among queues** to guarantee an **even distribution of tasks**:

- By balancing we can **better utilize CPUs and improve performance**
- We can **distribute tasks** to have **even temperature distribution** (thermal management impacts power consumption, efficiency and reliability)

Moving a task to another queue can be implemented in **two ways**:

1. **Push model**: a **dedicated task** periodically checks the queues' lengths and moves tasks if necessary
2. **Pull model**: each **processor** notifies an **empty queue condition** and picks tasks from other run-queues
 - **Work-stealing** is an example of pull-model approach. **In theory** is very **scalable**, but it **needs locking on queues** and an **algorithm to determine which queue to steal from**

A queue can also be **structured hierarchically**: we have a **global queue** and **many local ready queues**. This allows **better control over utilization and balancing** with good scalability, however it **very difficult to implement**.

Scheduling in Linux

Runqueues

The **central data structure** of the scheduler is known as the run queue. There is **one per CPU** in order to avoid contention over task selection.

Each runqueue has **different run queues for each scheduling class**.

Scheduling classes

A **scheduling class** is an **API** that includes **policy-specific** code to:

- **Update current task time statistics** (`task_tick`)
- **Pick the next task** from the queue (`pick_next_task`)
- **Select the core** on which the task must be enqueued (`select_task_rq`)
- **Put the task on that queue** (`enqueue_task`).

Scheduling classes allow developers to implement thread schedulers without reimplementing generic code and also helps minimizing the number of bugs.

Linux implements the following **classes**:

- **SCHED_DEADLINE**: implementation of the **Earliest Deadline First (EDF)** scheduling algorithm
- **SCHED_RR**: a task will **repeatedly go ahead of any task with lower priority** than itself; if multiple tasks have the **same priority**, it will **Round Robin** around those tasks
- **SCHED_FIFO**: like RR, but **if a task does not give up the CPU it will run indefinitely even if other tasks are the same priority** as itself
- **SCHED_OTHER** (also **SCHED_NORMAL**): **CFS** scheduler
- **SCHED_BATCH**: **longer timeslices**, well suited for batch jobs
- **SCHED_IDLE**: scheduled **only** if the CPU is **idle**

If **multiple policies have a runnable thread**, a choice must be made to determine which policy has the highest priority. Linux chooses a simple **fixed-priority list** to determine this order (deadline -> real-time -> fair -> idle).

The scheduler performs **load balancing by migrating threads between cores** in order to **even the number of threads of all cores**. Load balancing is done with a work stealing approach: **each core does its own balancing and tries to steal threads from the busiest core on the system**.

Ordering between processes is implemented with a priority π :

- $\pi \in [0; 99]$: **real-time** processes (**SCHED_FIFO** and **SCHED_RR**)
- $\pi \in [100; 39]$: **non real-time** processes (**SCHED_NORMAL**)
 - Priority of normal processes is a function of the **niceness** $v \in [-20; 19]$ and is calculated as $\pi = 120 + v$

CFS

See Love's Linux Kernel Development Ch. 4

Cgroups

CFS is not enough to guarantee optimal CPU usage, so we need a **mechanism to throttle and account for the CPU usage by various tasks**.

Main idea: **treat users as they were single tasks within the root runqueue and assign explicitly their own CPU weight**. Child tasks have their own runqueue and will take turn to consume each task group's timeslice. **These task groups are called cgroups**.

This is implemented through **groups that can recursively include other groups of tasks up to a root task group**. Each task group has a **dedicated CFS runqueue**.

Linux build a corresponding **hierarchy of schedule entities** (per CPU). When a task is accounted for time, also the parent group's entity is and so on until we arrive at the root entity. `__pick_next_entity()` **picks the entity with the smallest virtual runtime until a real task is selected**.

Load balancing

When you have $n > 1$ CPUs, making all CPUs do equal work (i.e., load balance) while respecting relative weights can get tricky:

- Can't balance on the same number of threads
- Can't balance on $\lambda_q = \sum_i \lambda_{i,q}$ of each run queue q .

The main idea is to **balance over the average load of a runqueue**, where the **load of a task i on q** is: $\omega_{i,q} = \lambda_{i,q} \gamma_{i,q}$ where $\gamma_{i,q}$ is the **CPU usage** of i on q .

Another thing to consider is that **it is not always worth it to move a task to another CPU**, e.g. due to **architectural reasons** (NUMA nodes, cache conflicts etc).

Linux, when it starts up, builds a **hierarchy of scheduling domains** (a scheduling domain is a **set of processing units that have some resources in common**) that **models the layout of the underlying CPU topology**. In the load-balancing algorithm, we call the **designated core** the core **that performs the load-balancing** (typically the first idle core). The hierarchy is typically seen relative to this core.

The **algorithm is periodically executed on the designated core**. It walks the hierarchy from the designated core **searching from work to steal**, i.e. search for the **busiest queue**. Once it finds it, **it pulls in tasks from that queue until balanced**.

IPC

Fork-wait

Forking is the operation whereby **a process spawns a new process which is a copy** of itself. When we call `fork()`:

1. A new process is created and runs concurrently
2. The virtual address space is copied: All the variables have the same value with the exception of the `fork()`'s return value
3. Most of the physical pages in the memory are marked as "CoW"

Every process has exactly **one parent** and may have an **arbitrary number of children**. The only exception is the **init process which is the ancestor of all processes**. Every process has a:

- PID: Process ID
- PPID: Parent Process ID

In Linux the PID is of type `pid_t`.

The `exec*` family of functions allow us to **load a new program** and replace the current process memory with it. The syscall they call is `execve`.

The **simplest synchronization** method between parent and child is the `wait()` function.

```
#include <sys/types.h>
#include <sys/wait.h>

// Suspend execution until one of the children exits
// Param:
// - status: pointer to a variable where to return the exit status of the child
//
pid_t wait(int *status);

// Suspend the execution until a specific child process terminates (or changes
// state)
// Param:
// - pid: pid of the child to wait
// - status: pointer to a variable where to return the exit status of the child
pid_t waitpid(pid_t pid, int *status, int options);
```

When a **process terminates**, the Linux kernel changes the **state to "zombie"** and **saves its return value**, which is **returned to the parent when the parent calls `wait()` (or `waitpid()`)** on the child process.

If the **parent terminates before calling wait-ing**, the child is **adopted by init**. The `init` process wait-s on all children freeing the memory and the PID number.

Signals

A signal is an **asynchronous, unidirectional “message” with no data transfer**. The **only information sent is the signal type**. Signals are used for **communication between processes** (not threads). They can be sent either by other processes or by the OS.

Most common signals:

SIG	Number	Action	Description
SIGHUP	1	Terminate	Terminal disconnected
SIGINT	2	Terminate	Terminal interrupt
SIGILL	4	Terminate and dump	Illegal instruction
SIGABRT	6	Terminate	Process abort signal
SIGKILL	9	Terminate	Process killed
SIGSEGV	11	Terminate and dump	Segmentation fault
SIGSYS	12	Terminate and dump	Invalid syscall
SIGPIPE	13	Terminate	Write on a pipe with no readers
SIGTERM	15	Terminate	Process terminated
SIGUSR1	16	Terminate	User-defined
SIGUSR2	17	Terminate	User-defined
SIGCHLD	18	Ignore	Child process terminated, stopped or continued
SIGSTOP	23	Suspend	Process stopped

```
#include <signal.h>
#include <sys/types.h>

// Send signal to process
int kill(pid_t pid, int sig);

// Specify how to handle a signal
// Params:
// - signum: signal to catch
// - act: new settings
// - oldact: output variable - old settings
int sigaction(int signum, const struct sigaction *act, struct sigaction *oldact);

struct sigaction {
    void (*sa_handler)(int);           // handler function
    void (*sa_sigaction)(int, siginfo_t *, void *); // alternative handler,
                                           // available with POSIX
                                           // real-time extension
    sigset_t sa_mask;                 // mask to block certain signals
    int sa_flags;                      // various options
    void (*sa_restorer)(void);         // DO NOT USE
};
```

The POSIX real-time extension adds some new functionality:

- `sigqueue()`: send a queued signal
- `sigwaitinfo()`: synchronously wait a signal
- `sigtimedwait()`: synchronously wait a signal, with timeout
- the `sa_sigaction` field allows us to specify a handler that accepts input data; flags must contain `SA_SIGINFO`

A signal can be masked to avoid disruption: it is similar to `SIG_IGN`, but the **signal is not dropped and is instead enqueued to be managed later** when the signal is unmasked. `SIGKILL` and `SIGSTOP` cannot be masked.

```
// Mask a signal
// Params:
// - how:
```

```
// - SIG_BLOCK: add to the mask
// - SIG_UNBLOCK: remove from the mask
// - SIG_SETMASK: replace the mask
// - set: set of signals
// - oldset: output variable - previous set of signals
int sigprocmask(int how, const struct sigset_t *set, struct sigset_t *oldset);
```

Unnamed pipes

Based on the **producer/consumer pattern: one producer writes, one consumer reads** (guaranteed by the OS in Linux). It is unidirectional. Data is written and read in **FIFO order**.

```
#include <unistd.h>
#include <fcntl.h>

// Create an unnamed pipe
// Params:
// - pipefd: array of 2 integers to be filled with two file descriptors:
// - [0]: fd for the read end of the pipe
// - [1]: fd for the write end of the pipe
// - flags:
// - O_CLOEXEC: close fds if exec*() is called
// - O_DIRECT: perform I/O in "packet mode"
// - O_NONBLOCK: avoid blocking in case of an empty pipe
int pipe(int pipefd[2]);
int pipe2(int pipefd[2], int flags);

// Low level read/write
ssize_t write(int fildes, const void *buf, size_t nbyte);
ssize_t read(int fildes, void *buf, size_t nbyte);

// Or we can transform our FS into a stream and use all stdio functions with it
FILE *fdopen(int fildes, const char *mode);
```

Named pipes (FIFO)

Same behaviour of unnamed pipes, but instead of working with raw fds, we are **using special files created by the OS**.
Note: no disk I/O is done, the file is just an abstraction.

```
#include <sys/types.h>
#include <sys/stat.h>
// Create a new named pipe
// Params:
// - pathname: name of the pipe
// - mode: permissions of the file (e.g. S_IWUSR for 0200)
int mkfifo(const char *pathname, mode_t mode);
```

Then we just read from the file like normal.

Message queues

IPC method suitable for **multiple readers and multiple writers**. Based on a **priority-queue where producers enqueue messages and consumers dequeue messages**. The **status of the message queue can be observable** (all files available in /dev/mqueue). Requires linking with the POSIX real time library (-lrt).

```
#include <mqueue.h>

struct mq_attr {
    long mq_flags;    // 0 or NON_BLOCK
```

```

    long mq_maxmsg; // max nr. messages in the queue
    long mq_msgsize; // max message size in bytes
    long mq_curmsgs; // nr. messages currently in the queue
};

// Open/create a message queue
// Params:
// - name: Unique name for the queue (starts with /)
// - oflag: Opening flags
// - mode: The file permissions to give to the file
// - attr: Attributes
mqd_t mq_open(const char *name, int oflag, mode_t mode, struct mq_attr *attr);

int mq_close(mqd_t mqdes);
int mq_unlink(const char* name);
int mq_getattr(mqd_t mqdes, struct mq_attr *attr);
int mq_setattr(mqd_t mqdes, const struct mq_attr *newattr, struct mq_attr *oldattr);

// Send a message
// Params:
// - mqdes: descriptor
// - msg_ptr: pointer to the message to send
// - msg_len: size of the message
// - msg_prio: priority of the message [0;31]
//   - higher msg_prio means higher priority
//   - messages of the same priority are handled in FIFO order
int mq_send(mqd_t mqdes, const char *msg_ptr, size_t msg_len, unsigned int msg_prio);

// Receive a message
// Params:
// - mqdes: descriptor
// - msg_ptr: output param - pointer to where to write the message
// - msg_len: size of the buffer
// - msg_prio: output param - priority of the read message
int mq_receive(mqd_t mqdes, char *msg_ptr, size_t msg_len, unsigned int *msg_prio);

```

Shared memory

Shared memory is an IPC mechanism that allows **two processes to share a memory segment**. In POSIX the shared memory is based on the **memory mapping** concept. It requires the linking to the POSIX real-time extension library (-lrt).

Like message queues, **opening/creation of shared memory segments are referenced by name**. In Linux a **special file is created under /dev/shm/<name>**.

```

#include <sys/mman.h>
#include <sys/stat.h>
#include <sys/fcntl.h>
#include <unistd.h>
#include <sys/types.h>

// Open/create a shared memory area
// Params:
// - name: unique name (starting with /)
// - oflag: opening flags
// - mode: the permissions
// Return: a file descriptor
int shm_open(const char *name, int oflag, mode_t mode);

```

```

// Setting the size of the shared segment
// Params:
// - fd:    descriptor
// - length: length in bytes
int ftruncate(int fd, off_t length);

// Map the memory to the VA space of the calling process
// Params:
// - addr:  starting address (null to let the kernel choose a value)
// - length: size of the mapped segment
// - prot:  memory protection flags
// - flags: visibility of the updates w.r.t other processes
//   - MAP_SHARED: updates visible to other processes and carried out through an
//     update of the underlying file
//   - MAP_PRIVATE: CoW updates
// - fd:    file descriptor
// - offset: offset to skip from the beginning of the fd
// Returns: pointer to the mapped area
void *mmap(void *addr, size_t length, int prot, int flags, int fd, off_t offset);

// cleanup
int munmap(void *addr, size_t length);
int shm_unlink(const char *name);

```

Synchronization: semaphores

The wait()/waitpid() based approach is clearly very limited. POSIX provides **semaphores**:

- Semaphore **counter = 0**: **wait**
- Semaphore **counter > 0**: **GO**

When the counter is maximum 1, the semaphore is called binary and behaves similarly to a mutex.

Semaphores work with **two atomic functions**:

- wait(): **block until counter > 0, then decrement and proceed**
- post(): **increment the counter**

Similarly to pipes, POSIX semaphores can be **named or unnamed**. They require the -pthread flag.

```

#include <semaphore.h>

// Create an unnamed semaphore
// Params:
// - sem: output parameter - semaphore struct to init
// - pshared: 0 if shared among threads, otherwise shared among processes
// - value: initial value
// Returns: 0 on success, -1 on error
int sem_init(sem_t *sem, int pshared, unsigned int value);

// Destroy a semaphore
int sem_destroy(sem_t *sem);

// Create a named semaphore
// Params:
// - name
// - oflags
// - mode
// - value

```

```

// Returns: pointer to the semaphore object or SEM_FAILED in case of error
sem_t * sem_open(const char *name, int oflags);
sem_t * sem_open(const char *name, int oflags, mode_t mode, unsigned int value);

// Close and delete a named semaphore
int sem_close(sem_t *sem);
int sem_unlink(const char *name);

int sem_wait(sem_t *sem);
int sem_trywait(sem_t *sem); // non-blocking version of wait
int sem_timedwait(sem_t *sem, const struct timespec *timeout); // wait for timeout

int sem_post(sem_t *sem);

```

Concurrency

We have **concurrency** when our **program is composed by activities (a sequence of instructions) where they can execute in overlapping time periods without a specified order**. On the other hand, **parallelism** is when we run **multiple tasks simultaneously** on hardware that has multiple compute resources.

We have **several models** for dealing with concurrency **enabled by different technologies**:

1. **Thread execution model**, enabled by:
 - HW parallelism (multicore)
 - SW timesharing
2. **Lightweight execution models**, enabled by:
 - Languages with coroutines or generator
 - Event-based constructs, enabled by:
 - Continuation passing (callbacks)
 - Languages with `async/await`

Issues

Intuitively we can characterize a program with **two properties**:

1. **Safety** (correctness): we don't reach an error state and don't work with invalid data
 - To guarantee safety we need **mutual-exclusion**, i.e. when two threads cannot act on the same resource at the same time
2. **Liveness** (progress): eventually, we reach a final state

One of the main ways that we can **lose liveness** is due to **deadlocks**. **Another problem** we may encounter when dealing with concurrent tasks **on the scheduler side** is **priority inversion**.

Kernel space concurrency

Kernel concurrency is different from user space concurrency as it **involves managing and synchronizing multiple threads running in the kernel space**. This includes handling **interrupts**, **preemptive scheduling**, and managing **shared resources among different kernel components**.

We have **3 sources of concurrency**:

1. **Kernel preemption**: in a preemptive kernel, `schedule` can be invoked mid-kernel and switch to another thread still in kernel mode
2. **Interrupts**: An interrupt can occur asynchronously at almost any time, interrupting the currently executing code in kernel mode
 - Code that is safe from concurrent access from an interrupt handler is said to be interrupt-safe
3. **Multiprocessing**

Kernel preemption

From 2.6 linux became **optionally preemptive**; the preemptive points are:

- **At the end of interrupt/exception handling**, when `TIF_NEED_RESCHED` flag in the thread descriptor has been set (forced process switch)
- If a **task in the kernel explicitly blocks and calls `schedule()`** (planned process switch). It is however always **assumed that the code that explicitly calls `schedule()` knows it is safe to reschedule**

Atomic context refers to the places where, while it could be possible to admit a preemption point, **it is not safe to call the scheduler and switch to a different thread**. Typical atomic context are when:

1. The kernel is running an interrupt or trap handler
2. The kernel is holding a spin lock
3. All programmers' defined places where it is not safe to preempt

`preempt_count` is the **variable used for controlling when a task in kernel mode can be preempted**:

1. Every time a task **acquires a lock**, we **increase** `preempt_count`; when we release a lock we decrement it
2. If `preempt_count == 0`, it is **safe** to switch to another task

The **real-time patch** makes **all kernel functions preemptible**, even interrupt handling.

Synchronization

If we had a uniprocessor machine, atomic context and interrupt enable/disable would be enough to ensure mutual exclusion. On SMP machines, however, we need an explicit mechanism. The spinlock is the primitive used.

A spinlock **continuously polls the lock until unlocked**. They are generally **useful when contention rates and critical section length are small** and/or you **can't afford the overhead** of a sleeping lock. Spinlock do not support recursion (the same thread trying to take the same lock multiple times).

To lock a section with a spinlock we can do:

```
DEFINE_SPINLOCK(mr_lock);
spin_lock(&mr_lock);
/* critical region ... */
spin_unlock(&mr_lock);
```

Spinlocks can be **implemented with a generic atomic operation** called “**compare and swap**”. The pseudocode for this instruction is the following:

```
T _atomic_compare_xchg(T *ptr, T old, T new) {
    T a = *p;
    if (a == old) *p = new;
    return a;
}
```

The spinlock can then implemented as such:

```
int lock;
while (_atomic_compare_xchg(&lock, 0, 1)) {};
```

rwlock

Readwrite locks are still **spinlocks**, but allow for **unlocking only reading/writing** and come with **irq save/restore variants**.

```
DEFINE_RWLOCK(mr_rwlock);
read_lock(&mr_rwlock);
/* critical section (read only) ... */
read_unlock(&mr_rwlock);

write_lock(&mr_rwlock);
```

```
/* critical section (read and write) ... */
write_unlock(&mr_lock);
```

seqlock

A seqlock allows **writers to push through concurrent reads** and, at the same time, **avoid to use locks if there are no concurrent writes**. Writers will not get blocked by concurrent reads (useful when writes are done in interrupt context).

```
write_seqlock(&mr_seq_lock); // increment seq. counter
/* write lock is obtained... */
write_sequnlock(&mr_seq_lock); // increment seq. counter

// -----
do {
    // V---- loops if seq. counter odd
    seq = read_seqbegin(&mr_seq_lock); // ^
    // read/copy data here ...          | check if seq. counter equal.
} while (read_seqretry(&mr_seq_lock, seq)); // V
```

jiffies, the variable that stores a Linux machine's uptime, is frequently read but written rarely by the timer interrupt handler; a seqlock is thus used for machines that do not have atomic 64 bit read. If seqlocks are used the function is the following:

```
u64 get_jiffies_64(void) {
    unsigned long seq;
    u64 ret;
    do {
        seq = read_seqbegin(&xtime_lock);
        ret = jiffies_64;
    } while (read_seqretry(&xtime_lock, seq));
    return ret;
}
```

Sleeping lock

They are **semaphores**, but in the kernel.

```
/* define and declare a semaphore, named mr_sem, with a count of one */
static DECLARE_MUTEX(mr_sem);
/* attempt to acquire the semaphore ... */
if (down_interruptible(&mr_sem)) { // we could use down(), but we wouldn't react
    // to any signal
    /* signal received, semaphore not acquired ... */
} else {
    /* critical region ... */
    /* release the given semaphore */
    up(&mr_sem);
}
```

up and down increase/decrease the semaphore. down_interruptible allows a waiting task to be woken up by a signal.

lockdep

If configured with CONFIG_PROVE_LOCKING=y, the kernel can be **run with a run-time mechanism for checking deadlocks**. This mechanism is called lockdep and detects violations of the following locking rules:

1. Locks acquired in **different order**
2. Spinlocks acquired in **interrupt handlers and also in process contexts when interrupts are enabled**

lockdep works with:

1. **Lock classes:** types of locks
2. **Lock instances:** instances of lock type (concrete locks)

It **keeps track of the lock instances state and dependencies**, checking the order of acquisition between the classes. If the order corresponds to that of a deadlock-able situation, it logs the situation.

Cache aware spinlocks

In SMP systems, **every attempt to acquire a lock requires moving the cache line containing that lock to the local CPU**. For contended locks, this cache-line bouncing can hurt performance significantly (**cache ping-pong**).

The general idea is the following:

1. When a CPU finds a **locked spinlock**, it **duplicates it in its cache, links** it with the one in the cache of the initial locker and **spins on the lock in its own cache**
 - This chain of links, we introduced a sort of **queue**
2. On **unlock**, the **unlocker gives the lock to the next in line**

This type of lock is implemented by the `qspinlock` structure.

In reality, cache-aware spinlock is implemented a bit differently. The structure is a 32-bit integer that is divided into the following **fields**:

1. **lock**: status of the lock
2. **pending**: used by the second locker for “optimistic waiting”
3. **tail**: pointer to the last lock in the queue

The **first process to wait for the lock doesn’t duplicate it**, but instead **spins on the same lock bit and sets pending** =

1. When a **third process** arrives, it **starts duplicating and building the queue, storing** in the central lock the **tail of the queue**.

Lock-free algorithms and data structures

Lock-free algorithms are vital in improving the performance of the Linux kernel. They **allow for synchronisation and concurrent access to shared resources without the need for locks**, which can cause contention and stall processing.

Per-CPU variables

The simplest way to reduce shared data is using **per-CPU variables**:

- A per-CPU variable is an **array of data structures, with one element per CPU** in the system, typically aligned to the size of the hardware cache
- **Each CPU can only access and modify its own element**
- Access to per-CPU variables should be **done with kernel preemption disabled** to avoid a CPU change during the variable manipulation.

```
#include <linux/percpu.h>
```

```
#include <linux/sched.h>
```

```
DEFINE_PER_CPU(int, counter); // array of struct
```

```
void do_something(void)
```

```
{
```

```
    /* Disables preemption as otherwise we could go to sleep and
    wakeup on a different cpu */
```

```
    cur_counter = get_cpu_var(counter);
```

```
    counter = counter + 1;
```

```
    /* Reenables preemption */
```

```
    put_cpu_var(counter);
```

```
}
```

Shared variables

There exists a Linux API to enforce the use **atomic read-modify-write instructions when available** in the ISA. It is based on `atomic_t` (32bit) and `atomic64_t` types and always **guarantees that such operations are not interruptible** by using under the hood `_atomic_compare_xchg`. Compare And Swap (CAS), however, is **not the silver bullet since it can still be fooled**.

The problem arises from the “**Compare**” **part**: as long as the **value involved in the comparison is the same**, the swap can proceed. Let us consider the following sequence on a linked list:

1. Thread 1: `pop()` but interrupted before the CAS

```
s.top -----+
              |
              V
T1.curh -> [ ]
              |
T1.nexth -> [ ]
              |
              |
```

2. Thread 2: `pop(); pop();`

```
s.top -----+
              |
T1.curh ->  X  |
              |
T1.nexth ->  X  |
              |
              [ ] <+
              |
```

3. Thread 3: `push();`

```
s.top -----+
              |
              V
T1.curh -> [ ]--+
              |
T1.nexth ->  X  |
              |
              [ ]--+
```

4. Thread 1: resumes at the CAS

```
s.top -----+
              |
              |
              V
              X
              |
              [ ]
```

This is a case of the so-called **ABA problem**.

Read-copy-update

RCU is a synchronization mechanism that allows for **lock-free read-side access to shared data** while ensuring **consistency with simultaneous writes**. It is a **deferred reclamation system** just like hazard pointers except that it is for anything happening in a critical section. The goal is to **provide the same low latency read performance to shared data that is frequently read and non-frequently written**.

The idea is for:

1. **Readers:**
 - Avoid locks

- Tolerate concurrent writes (might be multiple concurrent versions)
- **Might see old version for a limited time**

2. Writers:

- Are essentially **delayed**
- **Create a new version** (copy) of data structure
- **Publish new version with a single atomic instruction**

The reader uses `rcu_read_lock` to **inform the reclaimer that it is reading** and to create copies in case of writes. When they are finished they use `rcu_read_unlock`. A “**grace period**” must elapse between the two parts, and this grace period must be **long enough that any readers accessing the item being deleted have since dropped their references** (they entered the so called “**quiescent state**” and invoked `rcu_read_unlock`). This is called quiescent-state-based reclamation.

Since the read section cannot block, a **context switch is a quiescent state**. The kernel can **infer that a grace period elapsed when all other CPUs have executed a context switch**. The **reclaim callback is scheduled by the writer with `call_rcu` or `synchronize_rcu`**.

// READER

```
void manipulate_task_list(...) {
    rcu_read_lock(); // inform the reclaimer and disable preemption
    // cannot issue any blocking (sleeping) actions that might switch the context
    for_each_process(p) {
        /* Do something with p */
    }
    rcu_read_unlock();
}
```

// WRITER

```
void release_task(struct task_struct *p) {
    spin_lock(&tasklist_lock);
    list_del_rcu(&p->tasks); // removal phase. Must allow concurrent read/write access
    spin_unlock(&tasklist_lock);
    synchronize_rcu(); // wait for a grace period to pass
    kfree(p);
}
```

RCU used this way **synchronizes multiple readers and 1 writer**. If we have **multiple writers**, we need to use **proper locking**.

Memory consistency models

A memory model defines the **behavior of the visibility (and consistency) of the operations done by one thread of an SMP processor from another thread**. Put in another way, it determines **which values can be returned by read primitives**. A memory model is best understood in terms of possibilities that are excluded; essentially “if you write this, then the following cannot happen...”.

Due to write buffering, speculation and cache coherency protocols of modern processors, the order in which memory accesses are seen by another thread occur might be different from the order in the issuing thread.

Sequential consistency

We define a $<_p$, which defines a **program order** of the instructions in a single thread, and a $<_m$ which defined the **order in which these are visible in shared memory** (also called the *happens-before* relations). A multi-processor is called **sequentially consistent if and only if for all pairs of instructions** ($I_{p,i}, I_{p,j}$) you have:

$$I_{p,i} <_p I_{p,j} \implies I_{p,i} <_m I_{p,j}$$

In practice the operations of each individual processor appear in this sequence in the order specified by its program.

Total store order

This is the model that **Intel processors** use. When these processors issue a store, they utilise a **local write queue** - also known as a store buffer - to hide memory latency (a global lock is also used).

The **order of the stores is agreed upon by all processors** (due to the write queue). However, the **write queues are local to each processor**, so we can have **inconsistencies on reads** (which if possible **fetch the value in the queue**):

1. We can have **older writes appear after newer writes** (old write is still in the queue while the new has been flushed)
2. **Reads can read different values depending on the processor** (one read reads from the queue, while the other from shared memory)

To avoid inconsistencies, **“fence” instructions** are provided. These instructions simply **flush the write queue**.

Partial store order

It is the memory order of the **ARM processors** and is even **weaker than TSO**. Each processor **reads from and writes to its own complete copy of memory** (it is not a whole duplicate, but it is just a model). **Each write propagates** to the other processors **independently**, with **reordering allowed** as the writes propagate.

A small program that allows us to find out whether a machine uses a memory model or another is called a litmus test.

Hardware synchronization techniques

A data race is a **situation where the correctness of our program depends on the memory model and the ordering of threads**. In general, to **avoid data races** you want to reason more intuitively about the memory model behavior of TSO and PSO by **enforcing manually sequential consistency guarantees**. We can do this by using instructions such as fences or acquire/release.

1. **Fences (barriers): enforce the memory order to be equal to program order**. They **solve the ordering problems of TSO**.

They are pretty **costly** since they **require flushing the whole write queue** and stop the compiler from optimizing.

2. **Acquire/release**: they are two instructions:

- When we store-release, we want that **all necessary buffers are flushed before that store** (ensures that the release **happens after all write operations before it in program order**)
- When we acquire-load, we want that this **load is executed before all other instructions after it** (ensures that the **program order is respected for the load in question**)

Acquire/release is **lighter on resources** since it requires **flushing only the buffer we need and enforcing only the ordering strictly needed**, allowing the processor to still do some work/speculation in the meantime.

The linux kernel memory model (LKMM)

Compilers can introduce additional reordering of instructions that might appear as if the machine had a weaker memory model. This adds another layer of complexity to hardware memory models.

Higher level languages must give the programmer a way to enforce ordering happens-before relations just as it is done at the ISA level. This is called the **language memory model**. These ways **must work portably across SC/TSO/PSO platforms** and account for hardware differences. Moreover, **if we write a data race free program**, we will get that **it will behave like a sequentially consistent one, even exploiting hardware and software optimizations**.

In the kernel, the **primitives** `READ_ONCE (R[once])` , `WRITE_ONCE (W[once])` **prevent the compiler from reordering writes/reads, omitting them or doing too many of them**.

The **linux memory model** is the **least common denominator of all CPU families supported** by the kernel. Essentially, it is **PSO**. It is described as the **cumulative effect of a language-level model** (the subset of C specific to the kernel) **and the hardware models targeted** and describes the effect produced by special C instructions (or primitives).

For **synchronization**, the **primitives** `smp_store_release (W[release])` and `smp_load_acquire (R[acquire])` are used to create happens-before arcs between different threads.

Linux memory management

Every process has its **virtual address space**. Pages are then **mapped onto the physical space in the page directory**. The linux **kernel mappings are common to all processes** and **live on the upper range** of the address space. The kernel space is **divided** into two:

1. **Logical space**: logical mappings **map 1:1 in a continuous fashion** to the physical space from a known offset
 - In this area we can find, for example, the kernel code
 - Very easy to manipulate (simply add/remove an offset)
 - To allocate here we use `kmalloc(size)`
2. **Virtual space**: pages are **mapped in a non-contiguous way**
 - To allocate here we use `vmalloc(size)`

Process address space

In theory, a page table is all the kernel needs to implement virtual memory. However, **page tables represent only pages that are present in memory, not all the pages of a process**. How can we:

- Understand if a VPN is valid but not mapped?
- Get the corresponding data (disk, other existing pages in memory)?

Linux uses **Virtual memory areas (VMA)**. They are **basically virtual address ranges** of the process address space **with some flags that specify the permissions/behaviour** and **whether it is mapped to files or such**.

Many of them **correspond to file mappings** (executable or not) (**areas with a backing store**). Many VMA are **not mapped to any file** (e.g the stack and the heap), and we **call them anonymous**.

A **page fault** it is an **intervention by the virtual memory subsystem**. The **function** that is invoked on a page fault to **find and inspect the VMA** is `find_vma()`. Depending on the outcome we can have:

1. **Wrong permission**: causes a segmentation fault
2. **Unmapped address**: causes a segmentation fault
3. **Mapped, but not physically**: `handle_mm_fault` is called, **if a page table entry needs to be created**, `handle_pte_fault` uses the following functions:
 - `do_anonymous_page` to provide anonymous page for e.g. the stack
 - `do_swap_page`: read from backing store a page previously swapped out
 - `do_wp_page`: allocate a new anonymous CoW page for private mappings
 - `do_fault + filemap_fault`: read from file

VMA enable us to **do demand paging**. What is demand paging can be explained by looking at what happens on a `brk()`:

1. Program calls `brk()` to enlarge heap
2. `brk()` enlarges the VMA but does not allocate pages in physical memory
3. Process tries to access new memory: page fault happens and `find_vma` is called
4. `handle_mm_fault` and `handle_pte_fault` are invoked to build VPN to PPN mapping

Thus demand paging **means allocating physical memory only when needed**.

VMAs with a backing store can be **created explicitly** using `mmap()` with the backing store file descriptor:

```
int fd;
struct stat st;
char *addr;

// Open the file for reading
fd = open("test.txt", O_RDONLY);

// Get the size of the file
fstat(fd, &st)

// Map the file into memory, from here on, accessing
// *addr will access file's content
```

```
addr = mmap(NULL, st.st_size, PROT_READ, MAP_PRIVATE, fd, 0);
/// ...
```

In this way, we are given direct access to the kernel internal page cache!

Kernel address space

`kmalloc` is the main function we are going to use. It takes 2 arguments:

1. Size of the block to be allocated
2. **Allocation flags**, controls the behaviour of the function in a number of ways:
 - `GFP_KERNEL`: **normal allocation** of kernel memory, **may sleep**
 - `GFP_ATOMIC`: used to **allocate memory from interrupt handlers** and other code **outside of a process context**, **never sleeps**
 - `GFP_USER`: used to **allocate memory for user-space pages**, it may sleep

If there no more pages left, the **allocation may fail**. `kmalloc` “sleeping” means that **if there isn’t much memory, the kernel may put the process to sleep until there is more memory**.

`kmalloc` is **used for smaller allocations**. If we need **large chunks of contiguous pages**, we have three functions available:

```
get_zeroed_page(unsigned int flags);
_get_free_page(unsigned int flags);
get_free_pages(unsigned int flags, unsigned int order);
```

These functions can be **used at any time**, but **they may fail** due to lack of available memory. It’s also **crucial to ensure that the same number of pages allocated are freed**, or **else the system memory map gets corrupted**. Over-allocation can degrade system responsiveness as the kernel tries its best to fulfill allocation requests. This can potentially render the computer unusable or open new security vulnerabilities like denial-of-service.

If we want to have **precise control down to the NUMA node level** of where you want to allocate, you must **use a function called `alloc_pages_node`**:

```
struct page *alloc_pages_node(int nid, unsigned int flags, unsigned int order);
```

The `vmalloc` function is a Linux memory allocation mechanism that **allocates a contiguous memory region in the virtual address space**, returning a pointer to a linear memory area of size at least size or 0 if an error occurs. **Underlying physical pages are not contiguous** however. It’s **important that it’s not used for small allocations due to its overhead** and **shouldn’t be used in an atomic context because it could potentially sleep**.

All allocation functions have **corresponding functions to free** the allocated memory.

Kernel space allocation

In general, within the kernel, **small fixed size data structures are very often allocated and released**. We **cannot allocate a whole page just for a small object**, meaning we cannot use the previous functions (which deal in whole pages). The **kernel buffers its requests through two additional allocators**:

1. **Quicklists**: used only for paging
2. **Slab allocator**: used for other buffers

Basically, they are **pools of pre-allocated memory used for managing small object**.

The slab allocator

The slab allocator works with **frames, pages that are allocated to contain small data structures**. Each pool is accessed **through a `kmem_cache` object**, and there is **one for each type of data structure we want to allocate**. The **function used to allocate** in this structure is the following:

```
void *kmem_cache_alloc(struct kmem_cache *cachep, gfp_t flags);
```

For each `kmem_cache`, for each CPU we have a slab. This allows to **allocate memory without any spinlock** since it is **data local to each CPU**. If a slab fills up, it is **swapped out with a partially filled one** (`kmem_cache_node` keeps track of partially filled slabs). This **swapping requires, however, locking** since the slab may be associated with another CPU.

After a `kmem_cache_free`, the **object is left in an initialized state**, enabling a future allocation to start using the object immediately without wasting time initializing the structure.

The slab allocator provides **two main classes of caches**:

- **Dedicated**: These are **caches that are created in the kernel for commonly used objects** (e.g., `mm_struct`, `vm_area_struct`, etc...); they are reinitialised at free
- **Generic** (size-N and size-N(DMA)): These are **general purpose caches**, which in most cases **are of sizes corresponding to powers of two**

Under the hood, `kmalloc` also uses the slab allocator.

The buddy allocator

It is a **coarser-grain allocator** that **sits on top** of the slab allocator and the other allocator. The buddy allocator **handles**:

1. The **slab-allocator's pages**
2. The **`vmalloc`** requests
3. The **`alloc_pages`** requests
4. **User space pages**

For how it works, wikipedia has a very nice explanation.

Physical address space

Zonal page allocation

The system may be composed of different NUMA nodes. **Each node has fast access to the memory directly connected to it**; to access other memory it needs multiple hops to other CPUs. The **kernel is aware of the NUMA node layout** and **tries to account for it when allocating memory**.

The kernel **holds a list** (`pgdat_list`), with **one element** (`pg_data_t`) **for each NUMA node**. Each element is **subdivided into zones**. A **zone is basically a memory range that stores info on the pages that are in that range**. The **two most important zones** are:

1. `ZONE_DMA`: used for DMA
2. `ZONE_NORMAL`: used for everything else

Each zone contains **two structures**:

- `free_area`: **lists (one list of each order) of free contiguous page blocks**
- `watermarks`: parameters that are **used to regulate the free page usage**

User-space page caching

The **page cache is the set of physical page descriptors** (`struct page`) **corresponding to pages that contain data read and written** from regular filesystem files or associated with anonymous VMAs. It is accessible through:

1. **Forward mapping**: useful to **get to the physical page containing the file's data at an offset**
2. **Backward mapping**: useful **when we want to invalidate page tables entries of shared pages in different processes**

Some physical pages can be shared between VMAs for example because they are packed by the same file or are CoW. **Reverse mapping** allows to answer the question: **"given a page, find all VMAs that contain it"**. The page descriptor support reverse mapping with the following fields:

- `_count`: how many user space sharings there are
- `struct address_space`: how to reach the mapping in the case we want to invalidate it
- `flags`: describes the state of a page
- A reference counter for all the uses of the page

Page frame reclaim

We have seen that **each zone contains some watermarks**. These watermarks are an **indicator of how much free pages are there** in that zone (high, low, min). **Page reclaim** is handled by a **kernel thread called kswapd**.

1. If the zone has high, everything is normal
2. If the **zone is under high** (meaning it is low) kswapd is **started and periodically evicts pages asynchronously**
3. If the **zone is under low**, our **allocations trigger directly the kswapd process**
4. If the **zone reaches min**, the **allocator will start evicting pages**
 - **There can be cases (GFP_ATOMIC allocations) where pages can go under min**

The **page frame reclaim algorithm** is **derived from an algorithm called “clock algorithm”**. The **basic algorithm** goes like this:

- Keep a **circular list of pages in memory**
- We have a **cursor that goes along the list** (like the hands of a clock)
- Each **page has a reference bit**, which is **set if the page has been recently referenced** and is **automatically turned back to 0 after some time**
- If the **cursor finds a page with the reference bit set to 0**, that page is chosen to be **evicted**

This algorithm is very simple to implement and **efficiently approximates the LRU algorithm**, which is the optimal algorithm for this case. The true LRU algorithm requires using timestamps and timers, which are expensive.

The **in-kernel implementation** has a one **important tweak**: it **acknowledges the difference between anonymous and backed pages**. The optimization is based on the fact that **most often files are accessed only once and then never again**, it is roughly like this:

- The following is done both for anonymous and backed pages, with minor differences (we are going to see how it works for backed pages)
- We have **two lists**:
 - An **inactive list** containing **pages that have not been referenced for some time**, the eviction “**cursor**” **loops this list**
 - An **active list** containing pages that have been referenced recently
- **Pages need to be referenced two times to go back to the active list**
 - **Once** sets the reference bit to one and **moves the page to the front**, **twice moves the page to the active list**
- **Periodically the reference bit of pages in the active list is flipped to 0**
 - **Flipping the reference bit to 0 causes a move-to-front** of the page
- **Periodically the head of the active list will be moved to the inactive list**

System virtualisation

A **system virtual machine** is an efficient, **isolated duplicate of the real system that can run a commodity OS**. This is different than a process virtual machine, which is what we have seen up until now:

- Process virtual machine have the illusion of having the whole machine’s resources to themselves
- Process virtual machines, however, do not access peripherals directly, but through the operating system’s abstractions
 - This makes the virtualisation easier

It is **based on a virtual machine monitor**, or hypervisor, that relies solely on direct execution. The definition boils down to the following **requirements**:

- **Fidelity**: equivalence of behavior with the real machine
- **Safety**: the virtual machine cannot override the VMM’s control of virtualised resources
- **Efficiency**: programs should “show at worst only a minor decrease in performance”

Some terminology:

- **Host system**: the OS where virtual machines run
- **Guest system**: the OS that runs on top of the virtual machine
- **Virtual machine monitor/Hypervisor**: software program that translates or mediates access to physical resources such as interrupts or sensitive processor state
 - Ensures isolation

- **Type 1 Hypervisor**: also called native hypervisor; runs on bare metal without any OS abstraction
- **Type 2 Hypervisor**: runs in the context of another OS (think about KVM or VirtualBox).
- **Instruction types**: unprivileged and privileged. The latter are those that trap in user mode. The virtualisation idea is to run privileged instructions in a de-privileged mode.
- **Instruction sensitivity**: an instruction is virtualisation-sensitive if it is:
 - **Control sensitive**: it modifies directly the machine state (e.g., enabling or disabling interrupts, modifying the interrupt vector table)
 - **Behavior sensitive**: instructions that behave differently when used in either user or supervisor mode. It might affect fidelity.

This allows us to formulate the following **theorem** (from Popek and Goldberg):

For any conventional computer, a virtual machine monitor may be built if the set of sensitive instructions for that computer is a subset of the set of privileged instructions.

During the years, however, this theorem was **proven to not be 100% accurate**.

We can have **two types of virtualisation**:

1. **Software based virtualisation**: both **user and supervisor (OS) are run in user mode**, special instructions are “trapped and emulated” or binary translated
2. **Hardware based virtualisation**: the **CPU offers some hardware support** to reduce hypervisor interventions
 - CPUs with **virtualisation extensions** allow to **distinguish host user/supervisor modes and the guest user/supervisor modes**: the guest user and supervisor modes are running in guest-user and guest-supervisor modes, removing the need to collapse everything in user mode
 - **System calls need not to be trapped** any more, as the hardware routes them to the guest supervisor, we **need to intervene only when hardware is directly accessed** (since the guest sees only virtualised resources)

Software based virtualisation

Software-based virtualisation means **deprivileging**:

- The **guest supervisor is translated into host user mode** (e.g. x86 ring 1)
- **Privileged instructions or memory access produces an interceptable trap**
- The **host supervisor installs its own structures** (shadow structures) **instead of those dictated by the guest supervisor**

This mechanism becomes complicated when we analyze **how we handle the page table**:

- The **guest has its own page table**
- The **hypervisor needs to set up another level of indirection**: the **physical tables of the guest** (which are in reality the host virtual pages) **need to be re-mapped to physical pages on the host**
 - **Final map** installed by the hypervisor **needs to translate guest-virtual to host-physical**

This generates **a lot of interactions between supervisor and hypervisor**. Moreover, since we have **deprivileged both user and supervisor**, we can **have cases where the user might be able to read into the supervisor’s memory**, breaking the fidelity requirement. This means that **we also need to trap on supervisor range accesses to check permissions**.

Problems with pure trap-and-emulate on Intel machine

Originally, Intel had **unprivileged virtualisation sensitive instructions**:

- **Instructions manipulating the interrupt flags** (pushf, popf...): either **creating confusion in supervisor or hypervisor** as it cannot track the state of interrupts correctly.
- **Reading and writing segment descriptors and registers** (pop seg, push seg, mov seg, sgdt): the **supervisor can see that it has been deprivileged** by reading the Current Privilege Level and/or VMM state.
- **Excessive faulting**: On x86-32, sysenter and sysexit are used for each system call but trap into the VMM any time they are executed by the guest OS.

X86 CPUs also provide **many execution modes**, which means **higher complexity** in virtualisation. Between these modes, the **v8086 mode was strictly virtualizable** according to the Popek and Goldberg **theorem**.

Indeed, before the introduction of VMware, engineers from Intel Corporation were convinced their processors could not be virtualized in any practical sense.

Hardware assisted virtualisation

The goals are:

1. **Avoid** the **problems of deprivileging** by **adding new modes**
2. Allow the **state of the guest** to be explicitly and comprehensively **saved and resumed**
3. **Allow x86** to finally be **virtualisable according to Popek-Goldberg**
4. **Improve performance** by **reducing** the number of **traps** and **avoiding shadow-paging** overhead

It has been **implemented** in different ways by different vendors:

1. A **third mode** (hypervisor) in addition to user/supervisor (PowerPC, ARM)
2. A second **“dimension” for virtual machines** (x86, s390, RISC-V)
 - **RISC-V** has an **hybrid** approach:
 - **Guest code** runs in a **“Virtual User” mode**, while the **guest OS** in a **“Supervisor mode”**
 - **Host user code** runs in **normal “User” mode**, while the **host OS** runs in **“Hypervisor” mode**

Intel VT-x

Processors with this technology **basically duplicate all state of the processor: one copy for root mode, one copy for non-root (guest) mode.**

Every time a **hypervisor intervention** is needed, the **guest stops execution and dumps all the processor state into a hardware backed store (vmcs)** and **hands over control** to the root mode. The root mode has **instructions to query and modify** this structure.

Shadow paging is handled by **hardware assisted “extension”**:

- We basically have **2 cr3 registers**, one for the root and one for the non-root
- The **non-root only works with guest virtual pages**, the **root maps the guest virtual guest pages to the physical ones**
- The **hardware** then **composes** the two tables, obtaining the complete page table

This means that **page table coherency is done by hardware**, without hypervisor intervention. This **reduces significantly the number of hypervisor traps**. The drawback is, however, an **overhead on every TLB miss**.

KVM

It is an open source virtualisation technology built into the linux kernel that **allows Linux to function as a type 1 hypervisor** that provides memory management, scheduling, I/O etc. **Virtual machines are run as threads.**

```
// Super basic C-like VMM pseudocode
```

```
// Setup: interact with KVM from userspace
```

```
open("/dev/kvm");
```

```
ioctl(KVM_CREATE_VM); // Issue device specific commands
```

```
ioctl(KVM_CREATE_VCPU);
```

```
for(;;) {
```

```
    ioctl(KVM_RUN); // Switches to guest mode
```

```
    switch(vmcs -> exit_reason) { // Any event that causes a vmexit triggers a  
                                // resume of our application from here  
                                // Exit reason is stored in the vmcs
```

```
        case KVM_EXIT_IO:
```

```
            // Interrupted due to IO request
```

```
            // ...
```

```
        case KVM_EXIT_HLT:
```

```

    // Interrupted because the VM stopped execution
    // ...
}
}

```

QEMU can be used as a VMM by launching it with `--enable-kvm`. It provides a **solid ecosystem of emulated devices** and can provide fast access to them using the **VIRTIO interface**.

As **KVM VMs are processes**, **Linux virtual memory** allows by default to have each process consume all its address space, **enabling memory overcommitment**.

Balooning is a way to **enforce that some VMs actually stay with their memory usage under a certain threshold**. This is achieved by a **driver that reserves a certain amount of physical pages within the VM, forcing swapping**. Which pages are given back is the decision of the guest operating system.

Kernel same-page merging has the goal of **enabling sharing memory pages that have identical contents between multiple processes and virtualised guests**. It does this by **scanning physical pages that have identical content and identifying the virtual pages mapped to those physical pages**; then it **re-maps** all pages to one of the duplicates as **CoW** and then **releases the unused duplicates**. It was originally developed for KVM, but it can be used also for other things. Comparing whole pages can **consume a lot of computing power**, so use only when strictly necessary.

Containerization

Containers are a way to **isolate a set of processes and make them think that they are the only ones running** on the machine. The **machine they see may feature only a subset of the resources** actually available on the entire machine.

Containers are **not virtual machines**:

- **Processes** running inside a container are **normal processes running on the host kernel**
- There is **no guest kernel**
- You **cannot run an arbitrary operating system in a container**, since the **kernel is shared** with the host

Namespaces is the mechanism used to implement containers. It **provides a mean to segregate system resources**, similar to `chroot`. Processes have a pointer to a table (one for each namespace) that translates the namespace-local PID to OS PID.

IO and peripherals

Logically, memory and most peripherals are **accessible through a shared system bus**. **Device registers are exposed in the address space of the CPU**, where the processor can read/write commands/data (memory-mapped IO). **Some peripherals are accessible through logically different bus** (port bus); for this type of access we **need specific instructions** (port-based IO).

The **physical implementation is different than what we see logically**. We have **multiple point-to-point connections** to memory or different peripherals like graphics cards. We could also be **required to do different hops to reach a device**, passing through e.g. the IO chipset. Common bus types are: PCIe, SATA, USB.

CPU to device interactions

For **port-based IO**, IO is **done through explicit IO instructions**, for example on Intel we have `in` and `out`. Those are **usually privileged instructions**. Each **device is assigned a port number in the IO address space which names the device**.

Example: UART. To read more about what is go to Wikipedia.

The device exposes some control register and a Receive (RBR) 8-bit register and a Transmit (THB) 8-bit register.

To read from the device we can use `in`

```
in RBR, %al
```

For **memory-mapped IO**, IO ops are **done implicitly through loads/stores**. The **hardware makes the device registers available as if they were memory locations**.

Instructions such as in/out are **inefficient and limited by their use of registers**. Additionally, there is a **restriction of only 2^{16} port numbers**. **Devices registers can be operated on with loads/stores to specific physical addresses** (also called memory mapped registers), making port-based IO less useful.

Device to CPU communication

Devices can communicate with the CPU in various ways, like polling, interrupts or DMA.

1. **Polling**: the **processor waits until the device requires attention**
 - Can result in **wasted processor cycles if the device is slow**, but it is very inexpensive if the device is fast
2. **Interrupts**: the **OS puts the calling process to sleep**, when the **device** is done it **sends an hardware interrupt**, causing the CPU to **jump into the OS and execute and interrupt handler**
 - Better if the device is slow
3. **DMA**: introduces an **additional device, the DMA controller**, that can **manage transfers between different devices and memory autonomously**, without CPU involvement.

Linux low-level IO

Before interacting with the device, **we need to first request access to it**. This ensures that **no other process can access the device while we have it**. We can do this in two ways:

1. `request_region(start, len, driver_name)`: request a **port range** that starts from start and goes for range
2. `request_mem_region(start, len, driver_name)`: for **memory-mapped IO**, does the same thing

After access is granted, depending on the type of IO we can go about it in two ways:

1. For **port-based IO** we can **use inw, outb, outl instructions** to write to directly to the ports.

If we want to **treat the region as memory-mapped** (e.g. for platform agnostic drivers) we **need to call `ioremap()` to map it in memory**.

2. For **memory-mapped IO**, we need to **call `ioremap()` to map the physical address in virtual space** and ensure other safe access precautions (disable caching etc). **The we can write to it by calling `writel()`**.

To **write to memory-mapped or port-based-but-mapped regions in a generic way we can call the `iowrite32()` function**.

After we are done with the device, we **need to unmap** (if mapped) the memory **and then release** the region.

Interrupt management

Interrupts are associated with a specific number, called “vector”. In linux **vectors range from 0 to 200**. Vectors are divided into **3 sections**:

1. [0; 19]: used for **NMI**
2. [20; 31]: **reserved**
3. [32; 127]: used by **external** interrupts

Interrupts arrive from the device identified by an IRQ number, which is architecture specific. IRQs are then **mapped by the PIC** (Programmable Interrupt Controller) **to the corresponding vector**.

The **Interrupt Description Table** (pointed to by the IDTR register) **contains entries with pointers to functions that handle that interrupt (ISRs)**. We of course can **register our own ISRs**.

When linux **receives an interrupt it starts a function**, called `do_irq(n)`, that **receives the interrupt vector**. This function **disables local interrupts, executes our action and then re-enables interrupts**. To register an action we have a dedicated API:

```
static irqreturn_t handler(int irq, void *mydata) {  
    // ...  
    // acquire locks on shared data (remember we are in an interrupt context!)  
    // read/write from peripherals through MMIO  
    // defer work
```

```

// release lock
return IRQ_HANDLED;
}
static int __init mydriver_init_module(void) {
// allocate space for mydata
ret = request_irq(irqnum, handler, flags, mydata);
// ...
}

```

What we **usually want to do in an ISR** is to **defer work**, since **what we can do in an interrupt context is very limited**. Deferring work also improves system responsiveness, since ISRs are run with all interrupts disabled, and we can also aggregate many small operations into a single one, reducing work. Interrupt management is **structured in two levels**:

1. **Top half**: executes a **minimal amount of work which is mandatory** to later finalize the whole interrupt management work
 - Runs in non-interruptible mode
 - Schedules some deferred work
2. **Bottom half**: **finalizes the work deferred** from a queue and executes them. The bottom half is **invoked in particular reconciliation points**.

Deferred work is **done by softIRQs**. They are of different types, some of them are:

1. HI_SOFTIRQ
2. TIMER_SOFTIRQ
3. TASKLET_SOFTIRQ: provide a simpler interface to run deferred work

SoftIRQs are **work that is never interrupted and might be executed simultaneously in parallel on different CPUs**. If we schedule our work as a **tasklet**, **linux guarantees that only one instance of a tasklet can run at any time**.

To **schedule a softIRQs** for later execution we **call** `raise_softirq()`. A softIRQ is **executed** by `do_softirq()`. This function is **called at reconciliation points, which are**:

1. When linux **exits from an interrupt context**
 - At this point, we are still uninterruptible, so the kernel **only executes a bounded number of softIRQs** to maintain responsiveness
2. The **remaining work is executed by kernel threads** called `ksoftirqd/n` (one for each CPU) that **sit idle and cleanup softIRQ queues**

Tasklets

Like we said, they have a simpler interface and the kernel ensures no more than one instance of them is running concurrently. It is a **one-shot deferral scheme**: if you schedule two, only one is ran. A tasklet is a **pointer to a function plus some data**; it is **represented by the kernel with a list of tasklet_struct**.

```

struct tasklet_struct {
    struct tasklet_struct *next;
    unsigned long state; /* 0, scheduled or running */
    // ...
    void (*func)(unsigned long);
    unsigned long data;
};

// To declare a tasklet we have a handy macro:
DECLARE_TASKLET(my_tasklet, my_tasklet_handler, my_data);
// ...
tasklet_schedule(&my_tasklet); // this is invoked by the interrupt handler to
                               // schedule it

```

Work queues

If our **deferred action must block, we cannot use tasklets** (remember that tasklets cannot sleep!). In this case **we can use a work queue**, which is a **schedulable entity that runs in process context and executes the bottom half of an interrupt routine**. It is a **general mechanism to submit work to worker kernel threads** called `name_of_workqueues/n` with one for each processor. To **create work** for the worker we have a handy macro:

```
DECLARE_WORK(work, void (*func)(void *), void *data)
```

To **schedule it** we can call `schedule_work(&work)`

Devices in Linux

Devices are divided into three categories:

- A character device is characterized by a character stream. Writing Or reading from the file has direct impact on the device itself; no Buffering is performed
- A block device is seen as a sequence of numbered blocks. Each block can be individually addressed and accessed through a cache (random access)
- A network device, essentially a sequence of packets

Devices are integrated into the filesystem as special files stored under `/dev`. Processes use the standard file syscalls (read and write) to interact with devices. The VFS system routes said syscalls to the driver's implementation of the syscalls.

Each driver has a "major device number" that identifies it. If a driver supports multiple devices (like two disks of the same type) each device has a "minor device number" that identifies it.

In legacy systems, devices were created statically, meaning we had multiple devices for all supported drivers. Today, thanks to the introspection provided by sysfs (a virtual files system that contains files representing the state of the system), we have udev that handles on-demand device creation/destruction and persistent device naming.

Block devices

For a process to interact with a block device (usually a disk) we need to go through different layers:

1. Virtual filesystem: handles all the open, read, write, close files operations
2. The mapping layer (filesystem specific) translates the file+offset representation into a block number
3. The VFS checks if the page is already mapped in the page cache, otherwise we need to fetch it using a Block Input/Output (abbreviated as bio) request
4. The block layer receives the bio requests and dispatches them to the device driver
 - The interface the device driver needs to implement is the Request Queue interface

The linux device model

The Linux kernel runs on several architectures and hardware platforms. We thus need a way to:

1. Maximize the re-usability of code between platforms
2. Manage device lifetime
3. Cleanly organize the code based on a framework/bus matrix

The basic components are:

1. Device: structure that encapsulates information about the device
 - Each type of device driver specializes the generic device
2. Device driver: a structure that provides all information about devices that the driver can handle
 - Specializations of the generic device driver are created for the specific implementation/bus the driver uses (e.g. `i2c_driver`, `pci_driver`, `platform_driver`)

The kernel has various ways to discover the list of connected devices on a certain bus (ACPI/DT). Each driver announces what types of devices it can handle to the bus. For each device registered, the bus will ask if the driver can handle the device (by matching the vendor/model IDs); if the driver can handle it, we probe the device with said driver. Probing sets up the device resources (e.g. IRQs etc). If the probe completes successfully, the device is now operational and the driver interfaces with it.

Kernel frameworks are just libraries that simplify configuration of certain categories of devices. Each device/driver is the intersection between the various frameworks and a given bus.

The platform bus framework

It is designed for system-on-chip (SoC) and embedded devices and manages non-easily-discoverable hardware components. It is defined in board-specific code or Device Tree.

The PCI bus framework

In normal systems (like desktops) we usually attach peripherals to the PCI bus.

Linux platform configuration and boot

When booting, the OS must know a lot of things to start working, like which devices are already present on the machine, how interrupts are managed or how many processors are present. Peripheral devices standards such as PCI are not enough to find and configure everything in a platform. Two standards have evolved to provide the kernel with all the data of a platform (comprising PCI data):

1. ACPI (Advanced Configuration and Power Interface): used mainly on x86 and other general purpose platforms
2. Device Tree: used mainly on ARM and on embedded platforms

ACPI

Jointly developed by Intel, Microsoft and Toshiba in 1996, it provides an open standard for operating systems to:

1. Discover and configure computer hardware components
2. Perform power management e.g. putting unused hardware components to sleep
3. Perform auto configuration
4. Perform status monitoring

This removes the need to include platform-specific code for every platform, allowing to ship a single binary kernel that supports multiple platforms.